



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Karina García Morales

Asignatura: Fundamentos de Programación

Grupo: 22

No de Práctica(s): Practica 12

Integrante(s): Roja Martínez Francisco Alexander

*No. de lista o
brigada:* 40

Semestre: 2026-1

Fecha de entrega: 25 de noviembre de 2025

Observaciones:

CALIFICACIÓN:

Lectura y escritura de datos

Objetivo:

El alumnado elaborará programas en lenguaje C que requieran el uso de archivos de texto plano en la resolución de problemas, entendiendo a los archivos como un elemento de almacenamiento secundario.

Introducción

Un archivo es un conjunto de datos organizados que se agrupan en unidades llamadas registros. Estos registros comparten el mismo tipo de información, forman parte del mismo contexto y se guardan de manera ordenada para poder utilizarlos después. En el lenguaje C es posible trabajar con archivos para leer información de ellos o escribir datos en ellos. Esto se logra utilizando las funciones especiales que proporciona la biblioteca incluida en el encabezado `stdio.h`.

Apuntador a archivo

Un apuntador a archivo funciona como un enlace que conecta las operaciones de Entrada/Salida (E/S) con un búfer donde se mueven los datos. Este apuntador identifica el contenido del archivo y también especifica algunas de sus propiedades, como su nombre, su estado y la posición donde se encuentra el lector dentro del archivo. En el lenguaje C, los apuntadores a archivo se manejan como variables de tipo `FILE`, el cual está definido en la biblioteca `stdio.h`. Para declarar una variable que actúe como apuntador a archivo se utiliza la siguiente sintaxis:

Abrir archivo

La función `fopen()` se utiliza para abrir un archivo y vincularlo con una secuencia que permita trabajar con él. Su forma general es:

```
FILE *fopen(char *nombre_archivo, char *modo);
```

Aquí, `nombre_archivo` es un apuntador a una cadena de caracteres que indica el nombre del archivo, y puede incluir también la ruta donde se encuentra. Por otro lado, la cadena `modo` especifica la forma en que el archivo será abierto.

Los archivos pueden abrirse de distintas maneras dependiendo de lo que se necesite hacer con ellos. A continuación se muestran los modos más comunes, y varios de ellos pueden combinarse:

- *r*: Abre un archivo de texto solo para lectura.
- *w*: Crea (o sobrescribe) un archivo de texto para escritura.
- *a*: Abre un archivo de texto para agregar información al final.
- *r+*: Abre un archivo de texto para lectura y escritura.
- *w+*: Crea un archivo de texto para lectura y escritura.
- *a+*: Permite leer y agregar información a un archivo existente, o crearlo si no existe.
- *rb*: Abre un archivo en modo lectura binaria.
- *wb*: Crea un archivo en modo escritura binaria.

Cerrar archivo

La función **fclose()** se utiliza para cerrar un archivo que previamente se abrió con **fopen()**. Cuando se ejecuta, se encarga de guardar en el disco cualquier información que todavía esté en el *buffer* y realiza el cierre oficial del archivo dentro del sistema operativo.

Si ocurre un error al cerrar el archivo, pueden aparecer muchos problemas, como pérdida de información, daños en el archivo o fallas ocasionales en el programa.

Su prototipo es: ***int fclose(FILE apArch);**

Aquí, *apArch* es el apuntador al archivo que regresó la función **fopen()**.

Si **fclose()** regresa **0**, significa que el archivo se cerró correctamente. Esta función casi siempre falla solo cuando el disco se retira antes de tiempo o cuando ya no queda espacio disponible para guardar la información.

Funciones fgets y fputs

Las funciones **fgets()** y **fputs()** se utilizan para manejar cadenas dentro de archivos: una para leer y la otra para escribir. Sus prototipos son los siguientes:

```
char *fgets(char *buffer, int tamaño, FILE *apArch);
```

```
char *fputs(char *buffer, FILE *apArch);
```

La función **fputs()** escribe una cadena en el archivo indicado por el apuntador *apArch*. Por su parte, **fgets()** permite obtener una cadena desde un archivo; esta función lee el

contenido línea por línea y lo almacena en buffer hasta un máximo de tamaño caracteres.

Funciones fscanf y fprintf

Las funciones `fprintf()` y `fscanf()` funcionan de manera parecida a `printf()` y `scanf()`, solo que en lugar de trabajar con la pantalla o el teclado, realizan sus operaciones directamente sobre un archivo. Sus prototipos son:

```
int fprintf(FILE *apArch, char *formato, ...);
```

```
int fscanf(FILE *apArch, char *formato, ...);
```

En ambas funciones, `apArch` es un apuntador al archivo que previamente se abrió con `fopen()`. Esto significa que `fprintf()` y `fscanf()` realizan la escritura y lectura en ese archivo específico.

El parámetro `formato` es una cadena que puede contener texto normal y especificadores de formato, igual que en `printf` o `scanf`. En los puntos suspensivos se colocan las variables cuyos valores se escribirán en el archivo (*en el caso de `fprintf`*) o donde se guardarán los datos leídos desde el archivo (*en el caso de `fscanf`*).

Funciones fread y fwrite

Las funciones `fread` y `fwrite` se usan para manipular datos cuando se trabaja con elementos de tamaño fijo.

`fread` permite leer uno o varios bloques de datos del mismo tamaño, tomando como referencia una dirección de memoria (un apuntador). Esta función devuelve la cantidad de elementos que logró leer. Su forma general es:

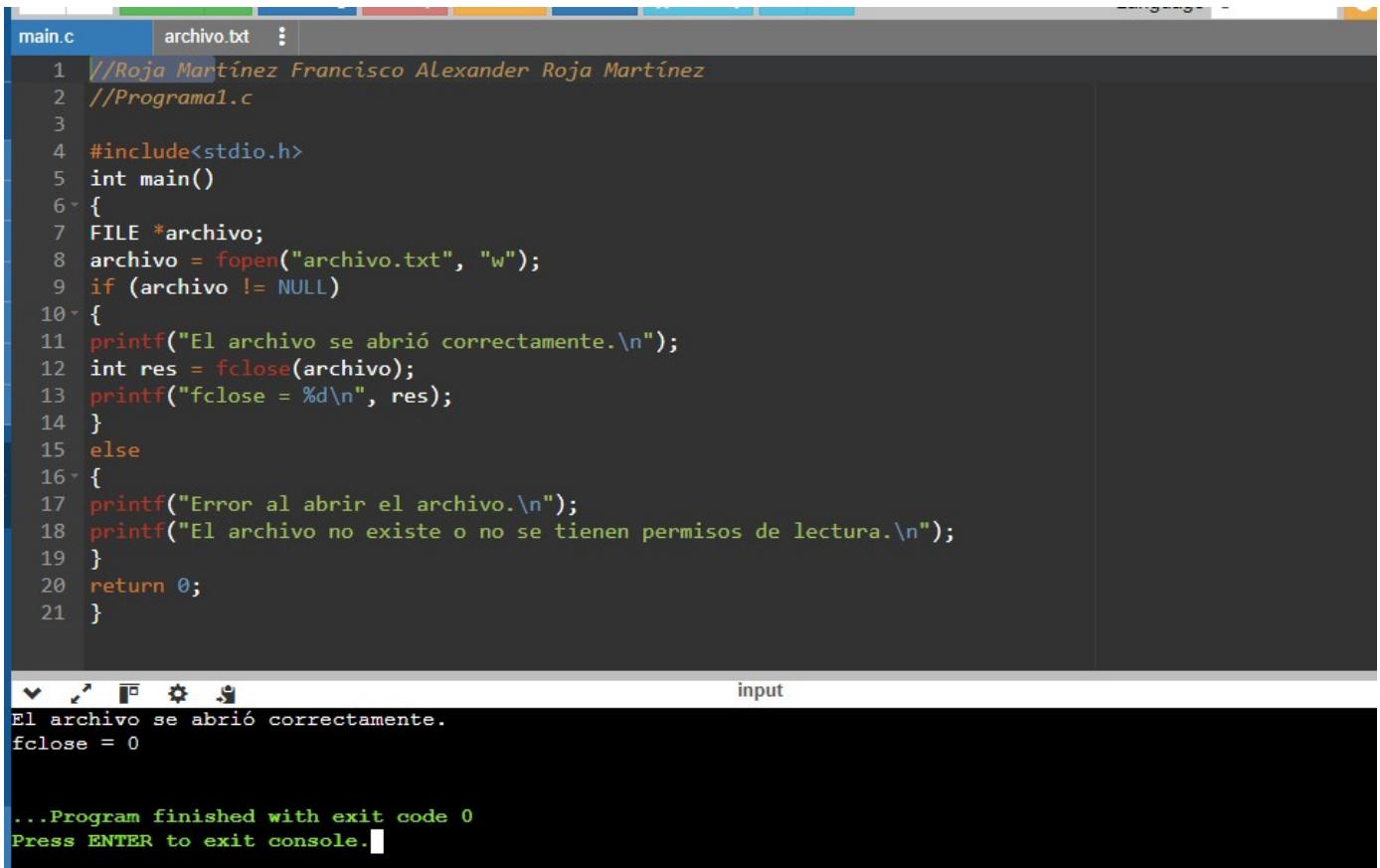
```
int fread(void *ap, size_t tam, size_t nelem, FILE *archivo)
```

Por otro lado, `fwrite` se encarga de escribir en un archivo uno o varios elementos que también tienen la misma longitud, tomando los datos a partir de una dirección de memoria. Su valor de retorno es la cantidad de elementos que se escribieron correctamente. Su sintaxis es:

```
int fwrite(void *ap, size_t tam, size_t nelem, FILE *archivo)
```

Ejercicos en clase

Programa 1.c



The image shows a code editor window with a file named 'main.c' and a file named 'archivo.txt' in the background. The code in 'main.c' is a C program that attempts to open a file named 'archivo.txt' in write mode ('w'). It uses 'fopen' to open the file, and if successful, it prints a message and then uses 'fclose' to close the file. If the file cannot be opened, it prints an error message. The program then returns 0. Below the code editor, there is a console window showing the output of the program. The output indicates that the file was opened successfully and that 'fclose' returned 0. The console also shows the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

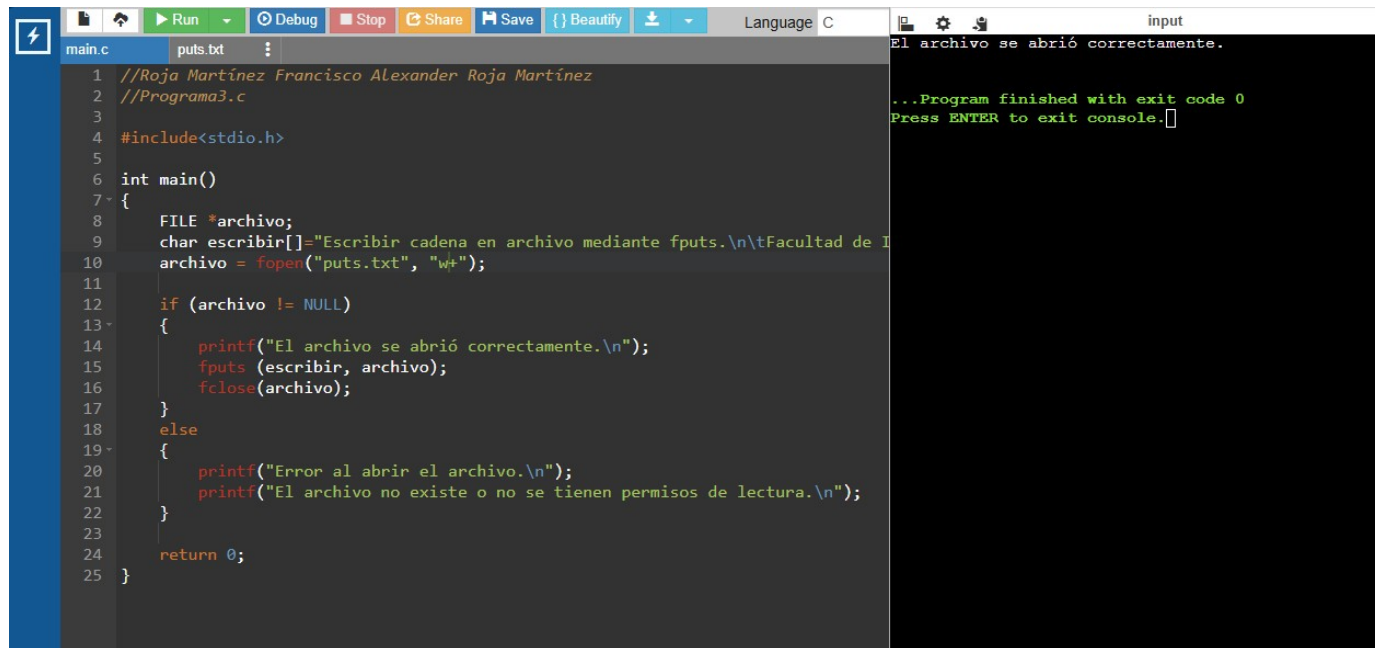
```
1 //Rojá Martínez Francisco Alexander Roja Martínez
2 //Programa1.c
3
4 #include<stdio.h>
5 int main()
6 {
7     FILE *archivo;
8     archivo = fopen("archivo.txt", "w");
9     if (archivo != NULL)
10    {
11        printf("El archivo se abrió correctamente.\n");
12        int res = fclose(archivo);
13        printf("fclose = %d\n", res);
14    }
15    else
16    {
17        printf("Error al abrir el archivo.\n");
18        printf("El archivo no existe o no se tienen permisos de lectura.\n");
19    }
20    return 0;
21 }
```

input

El archivo se abrió correctamente.
fclose = 0

...Program finished with exit code 0
Press ENTER to exit console.

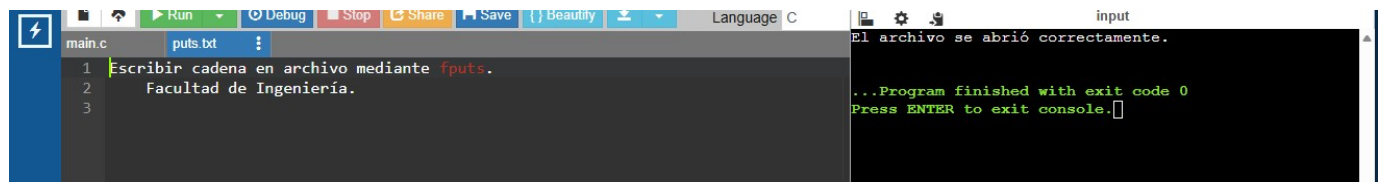
Programa 3.c



```
1 //Roja Martínez Francisco Alexander Roja Martínez
2 //Programa3.c
3
4 #include<stdio.h>
5
6 int main()
7 {
8     FILE *archivo;
9     char escribir[]="Escribir cadena en archivo mediante fputs.\n\tFacultad de I
10     archivo = fopen("puts.txt", "w+");
11
12     if (archivo != NULL)
13     {
14         printf("El archivo se abrió correctamente.\n");
15         fputs (escribir, archivo);
16         fclose(archivo);
17     }
18     else
19     {
20         printf("Error al abrir el archivo.\n");
21         printf("El archivo no existe o no se tienen permisos de lectura.\n");
22     }
23
24     return 0;
25 }
```

El archivo se abrió correctamente.

...Program finished with exit code 0
Press ENTER to exit console.

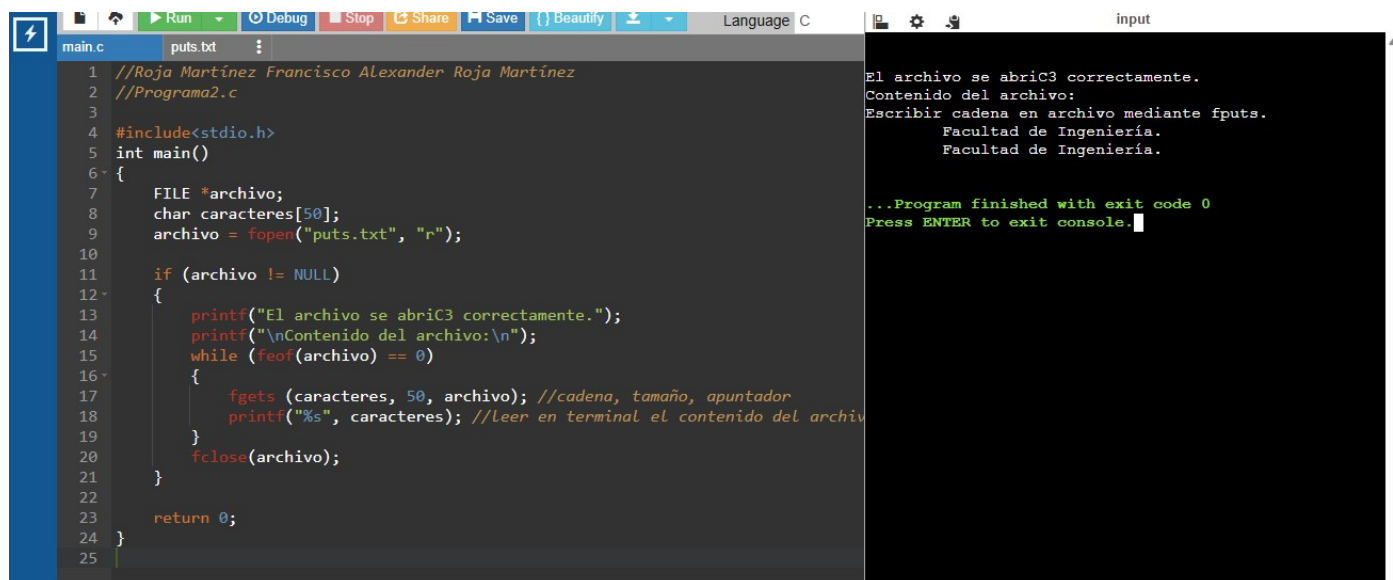


```
1 Escribir cadena en archivo mediante fputs.
2 Facultad de Ingeniería.
3
```

El archivo se abrió correctamente.

...Program finished with exit code 0
Press ENTER to exit console.

Programa 2.c



```
1 //Roja Martínez Francisco Alexander Roja Martínez
2 //Programa2.c
3
4 #include<stdio.h>
5 int main()
6 {
7     FILE *archivo;
8     char caracteres[50];
9     archivo = fopen("puts.txt", "r");
10
11     if (archivo != NULL)
12     {
13         printf("El archivo se abrió correctamente.");
14         printf("\nContenido del archivo:\n");
15         while (feof(archivo) == 0)
16         {
17             fgets (caracteres, 50, archivo); //cadena, tamaño, apuntador
18             printf("%s", caracteres); //Leer en terminal el contenido del archiv
19         }
20         fclose(archivo);
21     }
22
23     return 0;
24 }
25
```

El archivo se abrió correctamente.
Contenido del archivo:
Escribir cadena en archivo mediante fputs.
Facultad de Ingeniería.
Facultad de Ingeniería.

...Program finished with exit code 0
Press ENTER to exit console.

```
1 Escribir cadena en archivo mediante fprintf.
2 Facultad de IngenierC-a.
3 UNAM
4
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Programa 5.c

```
1 //Rojá Martínez Francisco Alexander Roja Martínez
2 //Programa5.c
3
4 #include<stdio.h>
5 int main()
6 {
7     FILE *archivo;
8     char escribir[] = "Escribir cadena en archivo mediante fprintf. \nFacultad de
9         archivo = fopen("fprintf.txt", "w+");
10     if (archivo != NULL)
11     {
12         fprintf(archivo, escribir); //escribe la cadena
13         fprintf(archivo, "%s", "UNAM\n"); //Escribir algo más
14         fclose(archivo);
15     }
16     else
17     {
18         printf("El archivo no existe o no se tiene permisos de lectura/escritura
19     }
20     return 0;
21 }
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

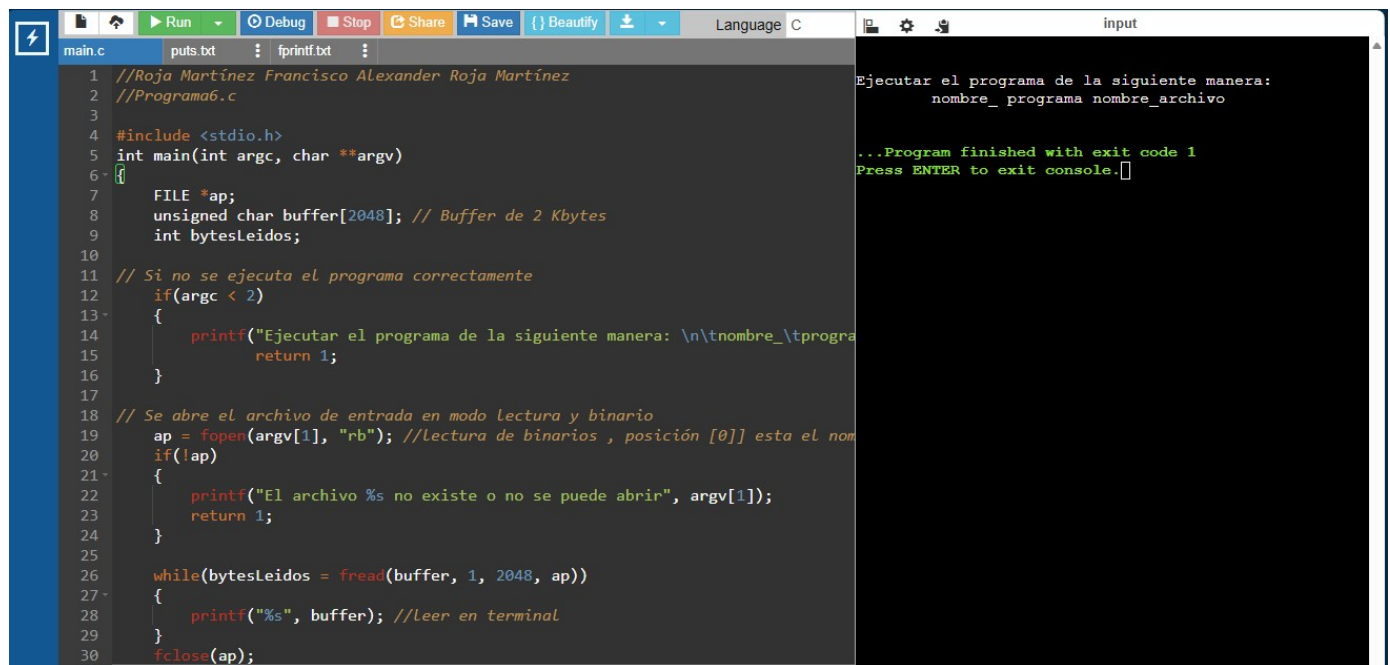
Programa 4.c

```
1 //Rojá Martínez Francisco Alexander Roja Martínez
2 //Programa4.c
3
4 #include<stdio.h>
5 int main()
6 {
7     FILE *archivo;
8     char caracteres[50];
9     archivo = fopen("fprintf.txt", "r");
10     if (archivo != NULL)
11     {
12         while (feof(archivo)==0)
13         {
14             fscanf(archivo, "%s", caracteres); //fscanf leer el contenido de un
15             printf("%s\n", caracteres);
16         }
17         fclose(archivo);
18     }
19     else
20     {
21         printf("El archivo no existe.\n");
22     }
23     return 0;
24 }
25
```

```
Escribir
cadena
en
archivo
mediante
fprintf.
Facultad
de
IngenierC-a.
UNAM
UNAM

...Program finished with exit code 0
Press ENTER to exit console.
```


Programa 6.c



The screenshot shows a C program editor with a dark theme. The left pane displays the source code for `Programa 6.c`, which is a file reader. The right pane shows the program's execution output in a terminal window.

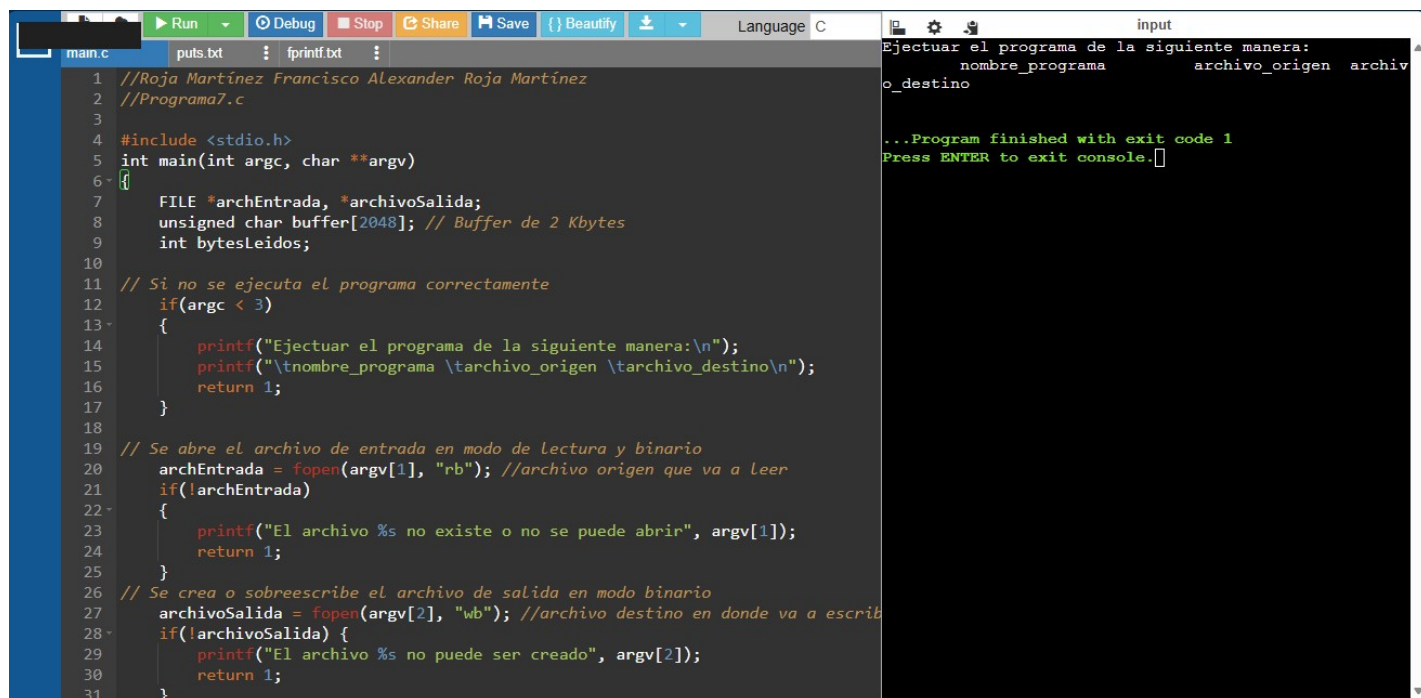
```
1 //Rojá Martínez Francisco Alexander Roja Martínez
2 //Programa6.c
3
4 #include <stdio.h>
5 int main(int argc, char **argv)
6 {
7     FILE *ap;
8     unsigned char buffer[2048]; // Buffer de 2 Kbytes
9     int bytesLeidos;
10
11 // Si no se ejecuta el programa correctamente
12 if(argc < 2)
13 {
14     printf("Ejecutar el programa de la siguiente manera: \n\tnombre\tprogra
15         return 1;
16 }
17
18 // Se abre el archivo de entrada en modo lectura y binario
19 ap = fopen(argv[1], "rb"); //Lectura de binarios , posición [0] esta el nom
20 if(!ap)
21 {
22     printf("El archivo %s no existe o no se puede abrir", argv[1]);
23     return 1;
24 }
25
26 while(bytesLeidos = fread(buffer, 1, 2048, ap))
27 {
28     printf("%s", buffer); //Leer en terminal
29 }
30 fclose(ap);
```

Execution output:

```
Ejecutar el programa de la siguiente manera:
    nombre_programa nombre_archivo

...Program finished with exit code 1
Press ENTER to exit console.
```

Programa 7.c



The screenshot shows a C program editor with a dark theme. The left pane displays the source code for `Programa 7.c`, which is a file copier. The right pane shows the program's execution output in a terminal window.

```
1 //Rojá Martínez Francisco Alexander Roja Martínez
2 //Programa7.c
3
4 #include <stdio.h>
5 int main(int argc, char **argv)
6 {
7     FILE *archEntrada, *archivoSalida;
8     unsigned char buffer[2048]; // Buffer de 2 Kbytes
9     int bytesLeidos;
10
11 // Si no se ejecuta el programa correctamente
12 if(argc < 3)
13 {
14     printf("Ejectuar el programa de la siguiente manera:\n");
15     printf("\tnombre_programa \tarchivo_origen \tarchivo_destino\n");
16     return 1;
17 }
18
19 // Se abre el archivo de entrada en modo de lectura y binario
20 archEntrada = fopen(argv[1], "rb"); //archivo origen que va a leer
21 if(!archEntrada)
22 {
23     printf("El archivo %s no existe o no se puede abrir", argv[1]);
24     return 1;
25 }
26
27 // Se crea o sobrescribe el archivo de salida en modo binario
28 archivoSalida = fopen(argv[2], "wb"); //archivo destino en donde va a escrib
29 if(!archivoSalida) {
30     printf("El archivo %s no puede ser creado", argv[2]);
31     return 1;
32 }
```

Execution output:

```
Ejectuar el programa de la siguiente manera:
    nombre_programa      archivo_origen  archiv
o_destino

...Program finished with exit code 1
Press ENTER to exit console.
```


Conclusión

Esta práctica me pareció interesante porque resultó más sencilla y menos complicada que otras que hemos realizado. Los únicos ejercicios que se me dificultaron fueron los últimos dos, especialmente al trabajar con la terminal, ya que todavía no entiendo muy bien cómo utilizarla. Aun así, pude completar los primeros ejercicios tal como la maestra los solicitó y los organicé en el mismo orden en que fueron indicados durante la clase.

Refereancia

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp.176-187 . Recuperado el 25 de noviembre de 2025 de [Laboratorio Salas A y B](#)